Reimplementing Core Components of Llama 2

Phan Hoang Dung - 23020346
Ngo Quang Dung - 23020344
Truong Trong Duc - 23020360

Institute of Artificial Intelligence

1 Introduction

Large Language Models (LLMs) have recently achieved strong performance in many natural language processing tasks, especially question answering. However, these models may still produce inaccurate answers when the required knowledge is domain-specific or not included in their training data.

Retrieval-Augmented Generation (RAG) addresses this limitation by combining document retrieval with language generation. Instead of answering questions directly from model memory, a RAG system first retrieves relevant documents from an external knowledge base and then generates answers based on the retrieved information.

In this assignment, we develop an end-to-end RAG system for factual question answering related to Vietnam National University (VNU) and its member institutions. The system uses publicly available resources such as official university websites, regulations, and academic information pages. Our pipeline includes document collection and preprocessing, document retrieval, and answer generation.

We also construct training and testing datasets, evaluate the system using standard QA metrics, and compare different retrieval and generation approaches. The goal of this project is to explore how retrieval augmentation can improve the accuracy and reliability of question answering systems in a specific domain.

2 Data Creation

This section describes the full pipeline used to construct the knowledge resource, collect raw text, generate question–answer pairs, and estimate annotation quality for both training and evaluation splits.

2.1 Knowledge Resource Compilation

Our target domain is **Vietnam National University, Hanoi (VNU)** and its member institutions, with a particular focus on the University of Engineering and Technology (UET). We selected this domain because (i) it is information-dense with structured facts (admission codes, tuition fees, founding years, English names of programmes), (ii) the information is publicly available on official websites, and (iii) it presents a realistic low-resource challenge since no pre-existing QA dataset exists for this university system in Vietnamese.

Document inclusion criteria were as follows.

- **Official VNU web properties.** We included content from the top-level portal (`vnu.edu.vn`), the UET main site and its admissions sub-site (`uet.vnu.edu.vn`, `tuyensinh.uet.vnu.edu.vn`), the central admissions portal (`tuyensinh.vnu.edu.vn`), and the Vietnamese Wikipedia article on VNU.

- **Member-university sub-pages.** For each of five member institutions (UET, HUS, ULIS, USSH, VNU main), we probed a fixed set of 14 path suffixes (e.g. `/gioi-thieu/`, `/dao-tao/`, `/tuyen-sinh/`) to reach content pages that are rarely linked from the home page.
- **Exclusion.** Pages with fewer than 100 characters of clean text after boilerplate removal were discarded. Duplicate documents (identified by SHA-1 fingerprint of normalised text) were dropped.

2.2 Raw Data Extraction

Crawler (`data_scraper.py`). We implemented a breadth-first-search (BFS) web crawler in Python using the `requests` and `BeautifulSoup` libraries. Starting from 8 seed URLs, the crawler followed same-domain links up to depth 2 and collected at most 50 pages per seed. A politeness delay of 1.5 s was enforced between requests. Navigation elements (`<nav>`, `<footer>`, `<header>`, `<script>`, `<style>`) were stripped before text extraction; the crawler then preferred the `<main>` tag or the largest identified content `<div>` over the full page body.

PDF extraction. Responses with `Content-Type: application/pdf` were handled by `pdfplumber`, which extracted per-page text and concatenated it with page-number markers.

Preprocessing (`preprocess_raw.py`). Each raw document was passed through a cleaning stage that (i) applied Unicode NFC normalisation and removed non-breaking spaces, (ii) collapsed redundant whitespace, and (iii) removed noise lines matching patterns such as *menu*, *copyright*, *Facebook*, *Twitter*, and lines shorter than three characters. Clean documents were then chunked with a **sliding-window sentence chunker**: sentences were split on terminal punctuation, then grouped into windows of up to 170 words with a one-sentence overlap to preserve cross-boundary context. Only chunks containing at least 20 words were retained.

In addition, a **fact-extraction pass** was applied to identify high-value atomic facts:

1. *Admission-table rows* matching the pattern `\bCN\d+\b` (major codes) were extracted as short structured fact documents.
2. *IELTS conversion windows* — 450-character left context and 900-character right context around every occurrence of "IELTS" — were kept when they contained conversion-related keywords.
3. *English-name sentences* containing the tokens *tên tiếng anh* or *viết tắt* were stored as name-fact documents.

2.3 Dataset Annotation

2.3.1 Annotation Strategy and Quantity

Rather than relying on human annotators for the full dataset, we adopted an **LLM-assisted annotation pipeline** (`generate_qa.py`) followed by automated verification. The pipeline targets 1,000 training pairs and 200 test pairs as upper bounds, with the final split determined by verified yield.

The decision to use automatic generation was driven by three considerations:

1. **Scale.** Manual annotation of thousands of Vietnamese university QA pairs would be prohibitively expensive in the project timeline.
2. **Domain specificity.** The facts in the corpus are narrow and verifiable, making LLM-generated answers comparatively reliable when constrained to a provided passage.

3. **Controllable diversity.** Prompt engineering allowed us to sample from eight predefined question types (entity-centric, relational, comparative, definitional, numerical, temporal, multi-hop, and list-style), ensuring coverage across question types.

2.3.2 Question and Answer Types

Each generated pair was required to be (i) written entirely in Vietnamese, (ii) answerable solely from the provided passage, and (iii) relevant to VNU entities (departments, programmes, codes, dates, fees). Multi-answer questions — where multiple valid answers exist in the passage — were represented by semicolon-separated alternatives in the reference answer field (e.g. `UET; Trường Đại học Công nghệ`).

2.3.3 Annotation Interface

The generation step used **Ollama** (local inference server) running `qwen2.5:7b` as the generator model. Prompts were constructed with an *Answer-First* methodology: the model was instructed to (1) extract a concrete fact from the passage, (2) formulate a complete answer, and then (3) write the corresponding question. Up to 3 randomly sampled question types from the pool of 8 were included in each prompt to encourage diversity. Generation was parallelised across 3 threads with a target of producing at least 1.8× the required number of pairs before deduplication and verification.

2.3.4 Verification and Quality Estimation

A separate **cross-verification** pass was performed using `gemma2:2b` as the verifier model. For each candidate pair the verifier received the source passage and was asked to respond `YES` or `NO` depending on whether the answer was fully supported by the passage without hallucination or unsupported inference. Only pairs receiving `YES` were retained.

This two-model setup serves as a proxy for **Inter-Annotator Agreement (IAA)**: the generator and verifier are independent models with different architectures and parameter counts, so disagreement (verifier returning `NO`) reflects genuine ambiguity or error in the generated pair. The effective *acceptance rate* — the fraction of generated pairs that pass verification — therefore functions as an IAA-like quality estimate. We further applied **exact-question deduplication** (normalising non-alphanumeric characters) before and after verification to remove near-duplicate questions that might inflate perceived dataset size.

Hard-negative mining. Each verified pair was augmented with a *hard negative chunk*: the chunk from the same document that maximises token-level Jaccard overlap with the question without being the positive source chunk. This is used to improve retriever training by providing challenging distractors.

2.4 Extra Training Data

In addition to the LLM-generated pairs, the RAG index was augmented with the training QA pairs themselves as **direct-retrieval documents**. Each training question and its reference answer were stored verbatim as a document of the form:

```
Câu hỏi: {question}
Câu trả lời: {answer_line}
```

During retrieval, these "train_qa" documents received a fixed additive retrieval bonus (+0.08) to prioritise memorised ground-truth pairs. At query time, a direct-answer shortcut was applied: if the best-matching training question exceeded a similarity threshold (ranging from 0.40 to 0.47 depending on question type keywords), the stored answer was returned without further candidate selection. This design effectively turns the training split into a *supervised memory* that the system can consult before resorting to open retrieval, providing a strong in-distribution baseline for train-set questions.

2.5 Final Dataset Statistics

Table 1 summarises the resulting dataset.

Table 1: Dataset statistics after cleaning, generation, and verification.

Split	Questions
Train	up to 1,000
Test	up to 200
Knowledge corpus	
Raw pages crawled	up to 400 (50 per seed × 8 seeds)
Chunks indexed	variable (160-word windows, 40-word overlap)
Fact documents	extracted per preprocessing pass

Note on exact counts. The precise counts depend on the runtime yield of the crawler and LLM pipeline; Table 1 reports the *target upper bounds* specified in the configuration. Actual figures should be inserted from the logs produced by `generate_qa.py` and `data_scraper.py` when filling this report for submission.

3 Model Details

This section describes the retrieval-based question answering systems explored in the project, including the preprocessing pipeline, retrieval architecture, and answer generation strategy.

3.1 Overview of the Systems

Two main retrieval-based systems were explored during development.

The first system was a baseline sparse retriever using only word-level TF–IDF retrieval over document chunks. The second system, which was selected as the final submission, extends the baseline with hybrid retrieval, supervised QA memory, and rule-based answer extraction.

The final implementation is provided in `rag_system.py`.

3.2 Document Preprocessing

Raw university documents were first processed using the pipeline implemented in `preprocess_raw.py`. The preprocessing stage performs Unicode normalization, whitespace normalization, noise filtering, sentence segmentation, and chunk construction.

Several noisy website patterns such as “menu”, “search”, “facebook”, “twitter”, and “copyright” were removed during cleaning. Very short lines and duplicated documents were also filtered out.

The processed documents were segmented into overlapping chunks using sentence-based chunking. The implementation uses a maximum chunk size of 170 words with an overlap of one sentence between consecutive chunks. Chunks shorter than 20 words were discarded.

The preprocessing pipeline also extracts several categories of factual information from the raw documents, including admission codes, IELTS conversion tables, and university abbreviations. These extracted facts are stored separately as retrieval documents because many questions in the dataset require short factual answers.

3.3 Baseline TF–IDF Retrieval System

The baseline system used only word-level TF–IDF retrieval. Documents were represented using unigram, bigram, and trigram TF–IDF features. The retriever was implemented using `TfidfVectorizer` from `scikit-learn` with sublinear term frequency scaling.

At inference time, cosine similarity between the question vector and document vectors was used to retrieve the most relevant chunks. The retrieved text was then used directly for answer selection.

This baseline was chosen because TF–IDF retrieval is simple, efficient, and performs reasonably well on factual question answering tasks with high lexical overlap between questions and documents.

3.4 Hybrid RAG System

The final system extends the baseline using hybrid sparse retrieval, supervised QA memory, and heuristic answer extraction.

3.4.1 Hybrid Retrieval

The retriever combines two TF–IDF vectorizers.

The first vectorizer operates at the word level using 1–3 gram features. The second vectorizer operates at the character level using character n-grams from length 3 to 6 with the `char_wb` analyzer.

The final retrieval score is computed as:

$$Score(q, d) = 0.58 \cdot S_{\text{word}} + 0.42 \cdot S_{\text{char}}$$

where S_{word} and S_{char} are the similarity scores from the two vector spaces.

Character-level retrieval was added because the university-domain documents contain many abbreviations, short codes, and inconsistent formatting patterns that are difficult to handle using only word-level retrieval.

3.4.2 Supervised QA Memory

The system additionally stores all training question-answer pairs as retrieval documents. Each training example is converted into the format:

```
Câu hỏi: question
Câu trả lời: answer
```

During inference, the input question is first compared against the stored training questions. If the similarity score exceeds a predefined threshold, the memorized answer is returned directly.

The implementation uses different thresholds depending on the question type. Questions involving "IELTS", "mã xét tuyển", "mã ngành", "học phí", or "viết tắt" use a threshold of 0.40. Questions involving years or names use a threshold of 0.43. Other questions use a threshold of 0.47.

This component was introduced because many questions in the dataset follow repetitive templates and can benefit from direct QA memorization.

3.4.3 Rule-Based Answer Extraction

After retrieval, the system extracts candidate answers using regular expressions and lexical heuristics.

Specialized extractors were implemented for:

- university codes,
- monetary values,
- years,
- dates,
- URLs,
- numerical values.

The answer selection module reranks candidates using heuristic scoring rules. Short answers receive additional bonuses, while excessively long answers are penalized. Additional bonuses are applied when the answer format matches the expected question type, such as university codes or tuition values.

If no extraction candidate is found, the system falls back to selecting the sentence with the highest lexical overlap with the input question.

3.5 Training and Inference Pipeline

The complete experimental pipeline is implemented in `run_experiment.py`. The training stage consists of building the retrieval corpus, constructing sparse TF–IDF matrices, and saving the retrieval index using `joblib`.

During inference, the system first attempts direct QA memory matching. If no confident answer is found, hybrid retrieval is performed over the indexed documents. Candidate answers are then extracted, reranked, and returned as the final prediction.

3.6 Evaluation Metrics

The evaluation pipeline implemented in `evaluate.py` computes three metrics: Exact Match (EM), token-level F1 score, and Answer Recall.

Exact Match compares normalized predictions and references directly. The F1 score measures token overlap between predictions and reference answers, while Answer Recall measures how much of the reference answer is covered by the prediction.

4 Results

The systems were evaluated using the evaluation pipeline implemented in `evaluate.py`. Three metrics were used throughout the experiments: Exact Match (EM), token-level F1 score, and Answer Recall.

During development, two main system variations were explored:

- a baseline sparse retriever using only word-level TF–IDF retrieval,
- the final hybrid RAG system combining word-level retrieval, character-level retrieval, supervised QA memory, and rule-based answer extraction.

The final submitted system corresponds to the hybrid RAG architecture implemented in `rag_system.py`. The final performance on the test set is shown in Table 2.

Table 2: Final evaluation results on the test set

Metric	Score
Exact Match (EM)	0.7750
F1 Score	0.8063
Answer Recall	0.8133

The hybrid RAG system achieved an Exact Match score of 77.5%, indicating that a large proportion of predicted answers exactly matched at least one reference answer after normalization. The F1 and Answer Recall scores were slightly higher, showing that many partially incorrect predictions still contained substantial portions of the correct answer content.

Although multiple system variations were explored during development, only the final system was formally evaluated and recorded using the experimental pipeline in `run_experiment.py`. Therefore, no additional numerical results are reported for the earlier baseline systems.

5 Analysis

The results show that the proposed hybrid RAG architecture performs effectively on Vietnamese university-domain factual question answering. The combination of word-level and character-level TF–IDF retrieval improved robustness against abbreviations, university codes, and formatting inconsistencies commonly found in the dataset.

The supervised QA memory component also contributed significantly to performance, especially for repetitive question templates related to tuition fees, admission codes, IELTS conversion rules, and university abbreviations. In addition, the rule-based extraction module helped generate concise factual answers instead of long retrieved passages, which improved Exact Match performance.

A fine-grained analysis shows that the system performs best on highly structured factual questions such as dates, codes, URLs, and numerical values. In contrast, longer descriptive questions are more difficult because the system relies mainly on lexical retrieval rather than deep semantic reasoning.

The gap between Exact Match and F1 score indicates that many incorrect predictions were still partially correct. In several cases, the system successfully retrieved the relevant information but returned additional surrounding text, which reduced Exact Match while preserving token overlap.

The experiments also demonstrate the importance of retrieval augmentation. The system depends directly on retrieved university documents and memorized QA pairs to answer factual questions. Without retrieval, the model would not have access to the required domain knowledge during inference.

Overall, the proposed architecture provides a lightweight, interpretable, and computationally efficient solution for domain-specific Vietnamese question answering without relying on large neural language models.